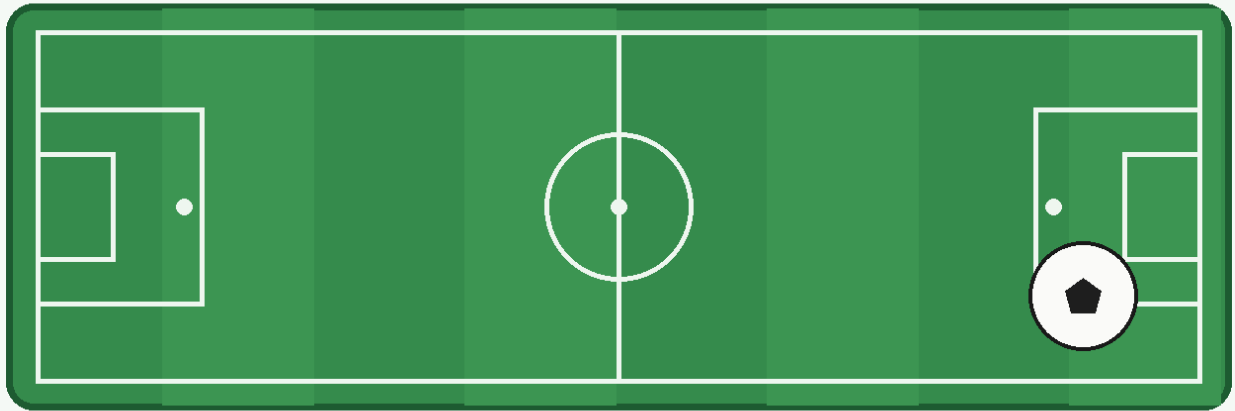


JOGO DE FUTEBOL

Tutorial Completo do Código

Arquitetura, lógica, IA, regras, física, renderização e controles



HTML • CSS • JavaScript • Canvas • Inteligência Artificial

Conteúdo

01. Arquitetura geral
02. index.html e composição da página
03. estilo.css e responsividade
04. app.js e entrada do sistema
05. configuracoes.js e constantes
06. Entidade Bola
07. Entidade Jogador
08. Atributos por função
09. Movimento, aceleração e fôlego
10. Estrutura de JogoFutebolWeb
11. Configuração de times e formações
12. Criação da partida e estatísticas
13. Loop principal e processamento
14. Entrada do usuário e ações
15. Passe e passe em profundidade
16. Cruzamentos
17. Finalizações e cavadinha
18. Dribles e proteção de posse
19. Desarme e carrinho
20. Faltas, vantagem e disciplina
21. Expulsões e limite de jogadores
22. Colisões e contato físico
23. Interceptações e bloqueios
24. Goleiros
25. Traves, cabeçadas e voleios
26. Impedimento
27. Detecção de regras e gol
28. Bolas paradas
29. Escanteios, faltas, pênaltis e laterais
30. Inteligência Artificial
31. Plano tático coletivo
32. Decisão com a bola
33. Passe inteligente e risco de interceptação
34. Defesa, pressão e cobertura
35. IA do goleiro
36. Substituições e mudanças táticas
37. Replay e comemorações
38. Áudio

- 39. Câmera 2.5D
- 40. Renderização do estádio e campo
- 41. Desenho dos jogadores e bola
- 42. HUD, minimapa e painel da IA
- 43. Estrutura dos controles de toque
- 44. Fluxo completo de uma jogada
- 45. Mapa de funções e linhas
- 46. Como estudar e ampliar o código

1. Arquitetura geral

O jogo foi organizado em arquivos com responsabilidades separadas. Essa divisão evita concentrar toda a lógica em um único ponto e facilita localizar regras, comportamento da IA, entidades físicas, interface e apresentação.

Arquivo	Responsabilidade principal
index.html	Cria a página, o Canvas da partida e os controles de toque.
css/estilo.css	Define layout, responsividade, aparência da página e botões móveis.
js/configuracoes.js	Centraliza medidas do campo, velocidades, tempos, raios, cores e utilitários matemáticos.
js/entidades.js	Implementa as classes Bola e Jogador, incluindo posse, movimento, atributos, fôlego e estados físicos.
js/ia.js	Implementa InteligenciaFutebol: plano tático, movimentação, passe, pressão, marcação, ataque e goleiros.
js/jogo.js	Coordena regras, placar, ações, bolas paradas, arbitragem, replay, câmera e renderização.
js/app.js	Liga teclado, mouse e toque à instância principal do jogo.
sons/	Armazena os efeitos sonoros locais.

Ideia central da arquitetura: entidades representam o que existe em campo; a IA decide comportamentos; JogoFutebolWeb coordena as regras e a apresentação; app.js conecta o navegador ao jogo.

2. index.html e composição da página

O arquivo index.html possui 279 linhas e funciona como ponto de entrada visual. O elemento principal é um canvas com id="jogo". Toda a partida, menu, estádio, jogadores, bola, placar e informações são desenhados nesse Canvas por JavaScript.

A página também contém a camada de controles de toque. Botões de direção usam data-tecla para simular teclas, enquanto botões de ação usam data-acao para chamar passe, profundidade, cruzamento, chute, chute forte, cavadinha, drible, proteção, desarme, carrinho, troca de jogador e substituição.

Os scripts são carregados de modo a disponibilizar primeiro configurações e classes auxiliares e, depois, a classe principal e a inicialização. Essa ordem é necessária porque JogoFutebolWeb depende de CFG, Bola, Jogador e InteligenciaFutebol.

3. estilo.css e responsividade

O arquivo `css/estilo.css` possui 356 linhas. Ele controla a moldura da página, centralização do Canvas, adaptação do tamanho à janela, aparência dos botões de toque e comportamento em telas menores. A física não depende do tamanho visual do Canvas: `app.js` converte as coordenadas do clique da dimensão exibida para a dimensão lógica definida por CFG.

Essa separação é importante porque o Canvas pode ser redimensionado pelo CSS sem alterar as coordenadas internas usadas por passe, colisão, impedimento ou IA.

4. app.js e entrada do sistema

O arquivo `js/app.js` possui 351 linhas comentadas e contém uma IIFE, isto é, uma função executada imediatamente. Ela busca o Canvas, cria `new JogoFutebolWeb(canvas)` e instala os eventos do navegador.

- `keydown` desbloqueia áudio, impede rolagem de teclas direcionais/Espaço, envia a primeira pressão para `teclaPressionada()` e mantém a tecla em `jogo.teclas`.
- `keyup` remove a tecla do conjunto, encerrando ações contínuas de movimento.
- `blur` limpa todas as teclas para impedir que uma tecla fique “presa” quando a janela perde foco.
- `pointerdown` no Canvas converte coordenadas da tela para o sistema lógico `CFG.LARGURA x CFG.ALTURA` e chama `clique()`.
- Os botões móveis adicionam/removem códigos no conjunto de teclas ou chamam diretamente métodos de ação.

5. configuracoes.js e constantes

O objeto CFG fica no início de `js/configuracoes.js`. Ele centraliza números que controlam o jogo. Essa estratégia evita espalhar valores importantes por dezenas de funções e permite compreender a escala do sistema em um único arquivo.

Grupo	Exemplos de constantes
Canvas e campo	LARGURA 1180, ALTURA 760, limites do campo, gol e área.
Movimento	VELOCIDADE_USUARIO 158, velocidades da IA por dificuldade, ACELERACAO 520 e MULT_SPRINT 1.24.
Fôlego	FOLEGO_MAX 100, GASTO_FOLEGO 23 e RECUP_FOLEGO 16.
Bola	VEL_PASSE 196, VEL_PROFUNDIDADE 258, VEL_CRUZAMENTO 270, VEL_CHUTE 438 e VEL_CHUTE_FORTE 635.
Disputas	ALCANCE_DESARME, COOLDOWN_DESARME, parâmetros de carrinho e probabilidade-base de falta.
Bola parada	DISTANCIA_BARREIRA, TEMPO_PREPARO_FALTA e TEMPO_COBRANCA_HUMANO.
Goleiro	TEMPO_GOLEIRO_COM_BOLA, TEMPO_MAX_GOLEIRO_MAOS e distância de segurança.

Disciplina	MIN_JOGADORES_5X5 3 e MIN_JOGADORES_11X11 7.
------------	--

No fim do arquivo existem utilitários matemáticos: `clamp()` limita valores, `dist()` calcula distância, `normalizar()` cria vetores unitários, `lerp()` interpola valores e `randomEntre()` sorteia dentro de um intervalo.

6. Entidade Bola

A classe Bola ocupa o início de `js/entidades.js`. O construtor guarda posição, velocidade, altura, velocidade vertical, curva, posse, tipo do último chute e referências importantes para regras e estatísticas.

``darPosse()`` prende logicamente a bola a um jogador e decide se a posse usa as mãos; ``soltar()`` remove a posse; ``chutar()`` recebe origem, alvo, velocidade, altura, impulso, curva, tipo e receptor; ``atualizar()`` aplica movimento, gravidade, atrito, curva, quique e comportamento de posse.

Método	Linhas	Função
Bola.constructor()	16-191	Cria o estado físico e lógico da bola.
Bola.soltar()	192-230	Remove posse e registra o último portador.
Bola.darPosse()	231-566	Entrega a bola ao jogador e trata uso das mãos.
Bola.chutar()	567-689	Calcula vetor, velocidade, altura, curva e metadados do lançamento.
Bola.atualizar()	690-1630	Atualiza física, trajetória e ligação com o portador.

7. Entidade Jogador

A classe Jogador começa na linha 1631 de `js/entidades.js`. Cada instância contém nome, time, número, posição, função, direção, velocidade, fôlego, cartões, estado de carrinho, estados de decisão da IA, atributos, aparência e dados usados por animações.

A quantidade de propriedades é proposital: um jogador não é apenas um círculo que se desloca. O código precisa lembrar se ele está protegendo a bola, cansado, caído, advertido, expulso, em carrinho, comprometido com uma corrida, aguardando nova decisão ou saindo do campo.

8. Atributos por função

``_criarAtributos()`` (linhas 2279-2542) começa com valores-base e substitui grupos conforme a função. Zagueiros recebem mais marcação, desarme, força e cabeceio; laterais combinam velocidade e marcação; volantes ganham proteção e passe; meias recebem passe, visão e domínio; pontas recebem velocidade e drible; atacantes concentram finalização e cabeceio; goleiros recebem reflexo e atributos próprios.

Uma pequena variação ligada ao número e índice do atleta evita que todos os jogadores da mesma função sejam idênticos. Os valores finais são limitados entre 35 e 95.

9. Movimento, aceleração e fôlego

``moverManual()`` (2589-2903) recebe direção, delta de tempo e indicação de sprint. A função normaliza a direção, calcula multiplicadores de atributo e fadiga, determina a velocidade-alvo e aproxima a velocidade atual desse alvo pela aceleração. O fôlego é consumido durante sprint e recuperado quando o atleta reduz esforço.

``moverPara()`` (2904-3559) cumpre papel semelhante para a IA, incluindo suavização de alvo, giro corporal e compromisso de movimento. ``atualizarEstadoFisico()`` (3671-4305) reduz temporizadores, trata carrinho, queda, expulsão e animações físicas.

10. Estrutura de JogoFutebolWeb

A classe `JogoFutebolWeb` começa na linha 8 de `js/jogo.js` e é o coordenador principal. Seu construtor cria contexto 2D, estado do menu, modo, dificuldade, duração, quantidade de jogadores, times, formações, IAs, sons, ambiente, estatísticas, árbitro, assistentes e inicia o loop com `requestAnimationFrame`.

O construtor também define quatro equipes com cores e perfis táticos, oito formações disponíveis e dois objetos `InteligenciaFutebol`, um para cada lado.

11. Configuração de times e formações

``_formacao()`` (1004-1991) devolve a lista de posições de cada atleta. Para 5 x 5, existe uma estrutura fixa. Para 11 x 11, o método contém os mapas de 4-3-3, 4-4-2, 4-2-3-1, 4-5-1, 3-5-2, 5-3-2, 5-4-1 e 3-4-3. Cada entrada informa nome, número, posição proporcional, se é goleiro e papel tático.

``_criarTimes()`` (2076-2309) transforma essas descrições em objetos `Jogador` e espelha posições para a equipe que ataca no sentido oposto. ``alternarTime()`` e ``alternarFormacao()`` mudam as escolhas do menu.

12. Criação da partida e estatísticas

``_zerarPartida()`` reinicia placar, relógio, mensagens, bola parada, vantagem, cartões pendentes, replay, câmera, táticas, árbitro, assistentes, substituições e uma estrutura detalhada de estatísticas. Entre os dados contabilizados estão chutes, chutes no gol, chutes fortes, gols de cabeça, voleios, cruzamentos, dribles, passes, interceptações, carrinhos, faltas, cartões, escanteios, impedimentos, defesas e posse.

``novaPartida()`` (2539-2601) aplica as escolhas do menu, recria times, reseta o estado e prepara o início efetivo do jogo.

13. Loop principal e processamento

``loop()`` (5696-5760) é chamado por `requestAnimationFrame`. Ele calcula o tempo transcorrido, limita valores extremos, chama `atualizar()` e `desenhar()`, e agenda o próximo quadro. ``atualizar()`` (5761-6521) coordena relógio, mensagens, física, IA, regras, câmera, replay, árbitro, assistentes, partículas e demais sistemas que precisam progredir no tempo.

A separação entre `atualizar` e `desenhar` é fundamental: a primeira modifica o estado lógico; a segunda apenas representa o estado atual no Canvas.

14. Entrada do usuário e ações

`_controleUsuario()` (6522-6636) lê o conjunto de teclas mantido por `app.js` e converte WASD/Setas em direção e Shift em sprint. `teclaPressionada()` (26834-27309) trata ações discretas como passe, chute, drible, pause, som e troca de jogador. `clique()` (27310-27626) trata o menu e interações por ponteiro.

15. Passe e passe em profundidade

`acaoPasse()` (6944-7053) escolhe a intenção do usuário e procura um alvo. `passar()` (8900-9507) calcula antecipação do movimento do receptor, força, direção, precisão e metadados usados por estatísticas e impedimento. `acaoProfundidade()` (7054-7128) chama `passeProfundidade()` (9508-9764), que usa velocidade maior e espaço futuro do companheiro.

A lógica mantém uma referência ao receptor esperado. Isso permite que outros sistemas reconheçam a intenção do passe e avaliem interceptação, domínio e possível impedimento.

16. Cruzamentos

`acaoCruzamento()` (7129-7278) decide a ação humana, incluindo combinação com Shift para cruzamento rasteiro. `cruzar()` (9765-10006) calcula alvo, trajetória, altura e receptor, registra estatística e prepara a bola para disputa aérea. A IA usa uma lógica complementar em `_alvoCruzamento()` dentro de `ia.js`.

17. Finalizações e cavadinha

`acaoChute()` (7279-7545), `acaoChuteForte()` (7546-7786) e `acaoCavadinha()` (7787-7944) transformam as teclas em tipos diferentes de finalização. O método `chutar()` de `JogoFutebolWeb` (10007-10427) calcula alvo, erro, velocidade, efeito visual, som, estatística e tipo do chute antes de enviar os parâmetros à Bola.

O chute colocado usa prioridade de direção; o forte usa velocidade muito maior e aumenta rebotes; a cavadinha usa elevação para tentar superar goleiro adiantado.

18. Dribles e proteção de posse

`acaoDrible()` (7945-8008) chama `tentarDrible()` (8009-8637). Essa função procura marcador próximo, escolhe um tipo de drible contextual, compara atributos e calcula deslocamento/resultado. O mesmo método pode ser chamado pela IA com parâmetros diferentes.

`acaoProtegerBola()` (5591-5673) ativa estados de proteção e pivô no Jogador. Esses estados são considerados por contato físico e decisões de passe.

19. Desarme e carrinho

`acaoDesarme()` e `acaoCarrinho()` são atalhos humanos. `tentarDesarme()` (10791-11287) avalia distância, direção, bola e contato; `tentarCarrinho()` (11288-11946) inicia o estado físico do carrinho e faz avaliação mais severa de contato e risco.

Na entidade Jogador, `iniciarCarrinho()` (3560-3670) define direção, velocidade, duração e animação. O estado continua sendo atualizado por `atualizarEstadoFisico()`.

20. Faltas, vantagem e disciplina

``_tratarInfracao()`` (11947-12170) recebe vítima, infrator e severidade. O método decide se a jogada pode seguir com vantagem e prepara cartão quando necessário. ``_atualizarVantagem()`` acompanha a janela em que a vantagem continua válida. ``_marcarFalta()`` (12874-12996) cria a reposição adequada, inclusive pênalti quando a infração ocorre na área.

``_aplicarCartao()`` (12281-12727) registra amarelo, segundo amarelo ou vermelho, atualiza estatísticas e inicia a retirada do atleta em caso de expulsão.

21. Expulsões e limite de jogadores

Depois de uma expulsão, ``_recompactarAposExpulsao()`` reorganiza o time.

``_verificarNumeroMinimoJogadores()`` (13552-13740) conta participantes válidos e encerra a partida quando o 5 x 5 fica com menos de três ou o 11 x 11 com menos de sete.

Na classe Jogador, ``_atualizarEstadoFisico()`` contém o movimento de saída do expulso: o atleta recebe alvo lateral, postura de choro e é retirado das coordenadas de jogo somente após concluir a caminhada.

22. Colisões e contato físico

``_resolverColisoes()`` (14521-14882) evita sobreposição excessiva e aplica resposta de contato entre atletas. Força, equilíbrio, proteção de bola, velocidade e direção alteram o deslocamento. O objetivo é permitir disputa corporal sem transformar jogadores em objetos que atravessam uns aos outros.

23. Interceptações e bloqueios

``_bloqueiosDefensivos()`` trata defensores colocados na trajetória de chutes. ``_interceptacoesAtivas()`` (15846-16311) procura defensores capazes de atacar linhas de passe e desviar ou dominar a bola. A IA também usa ``_riscoCorredor()`` e ``_vantagemIntercepcao()`` para evitar passes com alta probabilidade de corte.

24. Goleiros

``_defesasGoleiro()`` (15409-15845) avalia chutes e bolas perigosas próximas ao gol, considerando alcance, reflexo, posicionamento, força do chute e possibilidade de segurar ou espalmar. ``_reporGoleiro()`` (10428-10790) executa a saída após posse.

``_protegerPosseGoleiro()`` (13741-13981) impede adversários de invadirem a zona imediata do goleiro enquanto ele segura a bola. ``_garantirReposicaoGoleiro()`` (13982-14322) impõe limite de segurança e força uma reposição quando necessário.

25. Traves, cabeçadas e voleios

``_verificarTraves()`` (16312-16442) detecta contato da bola com postes e travessão e altera a trajetória.

``_cabeçadas()`` (16833-17716) organiza disputas aéreas e permite finalização ou corte; ``_voleios()`` (16443-16832) detecta situações em que a bola está em altura e posição adequadas para uma finalização no ar.

26. Impedimento

Antes de passes relevantes, `_registrarPossivelImpedimento()` (18368-18478) guarda contexto do instante do passe. `_estaImpedido()` (18479-18638) compara atacante, bola e linha de defensores. Quando a infração é confirmada, o jogo interrompe a jogada e sinaliza os assistentes.

A IA consulta a própria linha de impedimento ao decidir movimentos e passes, e a organização de bola parada procura manter atacantes em posições permitidas.

27. Detecção de regras e gol

`_verificarRegras()` (18639-19177) é um dos pontos centrais: verifica saídas pela linha de fundo ou lateral, posse anterior, escanteio, tiro de meta, gol e outras transições. `_gol()` (19178-19492) atualiza placar, estatísticas, replay, áudio, comemoração e prepara a futura saída de bola.

28. Bolas paradas

`_iniciarReinicio()` (19983-20223) cria um objeto que descreve tipo, time, posição e tempo de preparação.

`_organizarReinicio()` (20257-22263) posiciona cobrador, companheiros, adversários e goleiro.

`_montarBarreira()` (22463-22830) calcula a linha defensiva em faltas diretas, respeitando distância mínima.

`_atualizarReinicio()` (23308-23668) mantém o relógio e libera a cobrança humana ou automática.

`_pularBarreira()` anima o salto da barreira quando ocorre a finalização.

29. Escanteios, faltas, pênaltis e laterais

Sistema	Método	Linhas
Escanteio	<code>_cobrarEscanteio()</code>	24167-24298
Falta levantada	<code>_cobrarFaltaCruzada()</code>	24299-24437
Pênalti da IA	<code>_cobrarPenaltiIA()</code>	24438-24693
Lateral	<code>_cobrarLateral()</code>	24694-25214
Cobrança geral da IA	<code>_cobrarAutomatico()</code>	25215-25785
Alvo seguro	<code>_melhorPasseSeguro()</code>	25786-26025
Alvo de cruzamento	<code>_melhorAlvoCruzamento()</code>	26026-26274

Esses métodos usam as mesmas entidades e física da partida normal. A diferença é que primeiro existe uma fase de organização e restrições de posicionamento, depois a bola volta ao fluxo comum.

30. Inteligência Artificial

A classe `InteligenciaFutebol` começa em `js/ia.js` linha 8. Ela não controla somente o portador da bola. Cada ciclo recebe o time, adversários, bola, delta de tempo, jogo e informação sobre controle humano; então decide comportamentos individuais de acordo com plano coletivo, função e região.

``atualizarTime()`` (1325-1895) é a entrada principal. Ele atualiza o plano, identifica posse, goleiro, jogadores prioritários, pressão e papéis e encaminha cada atleta para atacar, defender, apoiar, buscar bola solta ou respeitar posse do goleiro adversário.

31. Plano tático coletivo

``_atualizarPlanoTatico()`` (290-763) considera posse, avanço da bola, placar, tempo restante e perfil da equipe. A saída é usada por outros métodos para determinar altura do bloco, intensidade de pressão, largura, ritmo de transição e risco permitido.

``_alvoEstrutural()`` cria um destino de referência a partir da posição base e do contexto, permitindo que a equipe bascule em torno da bola sem abandonar completamente a formação.

32. Decisão com a bola

``_atacarComBola()`` (4886-7160) reúne grande parte da decisão ofensiva. O portador avalia pressão, corredor de condução, qualidade de chute, possibilidade de drible, passe, profundidade, cruzamento, proteção e tempo mínimo de decisão. Há temporizadores para evitar que o agente troque de ideia a cada quadro.

A classe Jogador guarda histórico recente de escolhas e alvo anterior. ``_penalidadeRepeticao()`` desestimula repetir a mesma ação de forma artificial, enquanto ``_registrarEscolha()`` memoriza decisões recentes.

33. Passe inteligente e risco de interceptação

``_melhorPasse()`` (7608-8650) avalia candidatos usando progresso, espaço, direção, distância, papel, risco de corredor e valor ofensivo. ``_linhaPasseLivre()`` faz um teste geométrico; ``_riscoCorredor()`` mede proximidade dos adversários ao segmento; ``_vantagemIntercepcao()`` estima se um defensor chegaria antes que o passe complete o caminho.

O alvo não precisa ser o companheiro mais próximo. A pontuação pode favorecer mudança de corredor, progressão, terceiro homem ou jogador posicionado entre linhas.

34. Defesa, pressão e cobertura

``_selecionarPressionadores()`` limita quantos jogadores atacam o portador. Isso evita que o time inteiro se agrupe ao redor da bola. ``_atribuirMarcacoes()`` distribui alvos defensivos. ``_defender()`` (10639-11104) combina posição estrutural, cobertura, marcação e proteção da área; ``_pressionar()`` aproxima jogadores prioritários; ``_apoiar()`` mantém opções de passe quando a própria equipe possui a bola.

35. IA do goleiro

``_goleiro()`` (11369-12632) avalia posição da bola, ameaça de chute, cruzamento, espaço nas costas da defesa, necessidade de sair com as mãos, uso dos pés e posse. A função diferencia situações dentro e fora da área e respeita as regras de uso das mãos.

``_melhorSaidaGoleiro()`` (12633-13129) pontua companheiros para reposição, considerando distância, pressão, corredor de passe e risco. Se não houver opção segura, a lógica pode preferir uma saída longa.

36. Substituições e mudanças táticas

`_verificarSubstituicoesIA()` (4477-4704) analisa desgaste, cartões e momento da partida.
`_acaoSubstituicao()` (4705-4740) executa a ação manual quando permitida. `_verificarAjustesTaticosIA()` (5363-5590) altera formação e comportamento em partidas 11 x 11 conforme placar e tempo restante.

37. Replay e comemorações

`_registrarFrameReplay()` (4741-4878) armazena estados recentes em um buffer limitado.
`_iniciarReplayGol()` seleciona o trecho usado após o gol, e `_atualizarReplayGol()` avança o replay.
`_atualizarComemoracaoGol()` (19493-19880) controla movimentos do autor e companheiros durante a celebração.

38. Áudio

O construtor de `JogoFutebolWeb` pré-carrega efeitos WAV da pasta sons e cria dois ambientes em loop: menu e torcida. `_desbloquearAudio()` (2930-3035) tenta iniciar áudio após interação do usuário, requisito comum dos navegadores.

O grito de gol principal é carregado por URL externa. `_gritarGolReal()` tenta tocar o áudio;
`_gritarGolVozSistema()` serve como alternativa com a API de síntese de voz.
`_sincronizarAudioAmbiente()` alterna menu e torcida conforme o estado.

39. Câmera 2.5D

A física usa coordenadas 2D do campo. A sensação de profundidade nasce durante o desenho. `_atualizarCamera()` (26465-26833) escolhe zoom e deslocamento-alvo conforme bola, contra-ataque, bola parada, replay e comemoração. `_projetarCampo()` (30393-30511) converte coordenadas de mundo em coordenadas de tela e aplica escala ligada à profundidade.

Essa abordagem preserva regras simples de física e, ao mesmo tempo, produz perspectiva visual. O mesmo ponto lógico pode ser desenhado em tamanho diferente conforme a posição vertical no campo.

40. Renderização do estádio e campo

`_desenhar()` (27740-27868) escolhe qual tela deve aparecer: menu, ajuda, partida ou fim.
`_desenharPartida()` (29889-30392) compõe o quadro completo. `_desenharEstadio()` (30666-31425) cria arquibancadas, público, corredores e elementos externos; `_desenharCampo()` (31426-32144) desenha gramado, faixas, linhas, áreas e marcações; `_desenharGol()` acrescenta traves com profundidade e rede.

41. Desenho dos jogadores e bola

`_desenharJogador()` (32749-34745) usa a posição projetada, função, estado e animação para desenhar cabeça, tronco, braços, pernas, joelhos, meias, chuteiras e uniforme. A passada é ligada à velocidade e existem poses para sprint, passe, chute, chute forte, cabeçada, carrinho, proteção, queda e saída por expulsão.

`_desenharBola()` (34746-34991) aplica projeção, altura, sombra, rotação e efeitos associados a tipos de chute. Assim, a representação visual acompanha os dados físicos mantidos por Bola.

42. HUD, minimapa e painel da IA

`\_desenharHud()` (34992-35728) apresenta placar, relógio, nomes das equipes, informações de formação/tática e mensagens. `\_desenharMiniMapa()` (35729-36001) reduz as posições do campo para uma visão global. `\_desenharPainelIA()` (36002-36154) exibe informações internas quando o usuário ativa a tecla I.

43. Estrutura dos controles de toque

Os botões móveis são declarados em index.html, mas a lógica fica em app.js. Botões com data-tecla simulam pressionar e soltar uma tecla; botões com data-acao chamam métodos como acaoPasse(), acaoChuteForte() ou acaoCarrinho(). Essa organização evita duplicar a lógica do jogo para teclado e toque: as duas interfaces chegam aos mesmos métodos.

44. Fluxo completo de uma jogada

1. O navegador recebe uma tecla em app.js e a adiciona ao conjunto jogo.teclas ou chama teclaPressionada().
2. O loop de requestAnimationFrame calcula o delta de tempo e chama atualizar().
3. _controleUsuario() move o jogador ativo; as duas instâncias de InteligenciaFutebol decidem as ações dos demais atletas.
4. Quando uma ação de passe/chute ocorre, JogoFutebolWeb calcula intenção e chama Bola.chutar().
5. Bola.atualizar() move a bola; Jogador.atualizarEstadoFisico() mantém estados individuais.
6. O jogo resolve contato, bloqueios, interceptações, defesas do goleiro, traves, cabeçadas, voleios e posse.
7. _verificarRegras() verifica saídas, gol e reinícios; a arbitragem acompanha faltas, vantagem e cartões.
8. _atualizarCamera() define enquadramento; desenhar() transforma o estado em estádio, atletas, bola e HUD no Canvas.
9. O próximo quadro repete o processo, mantendo lógica e apresentação sincronizadas.

45. Mapa de funções e linhas

As faixas abaixo apontam para o arquivo enviado e facilitam localizar os sistemas diretamente no código. Como o projeto é extensamente comentado, os intervalos incluem comentários explicativos existentes entre as instruções.

Arquivo	Classe/Função	Linhas	Responsabilidade
js/configuracoes.js	CFG	10-499	Medidas, velocidades, tempos e cores
js/entidades.js	Bola	8-1630	Estado e física da bola
js/entidades.js	Jogador	1620-4364	Estado, atributos, movimento e animações
js/entidades.js	_criarAtributos()	2279-2542	Atributos por função
js/entidades.js	moverManual()	2589-2903	Movimento controlado
js/entidades.js	moverPara()	2904-3559	Movimento da IA

js/entidades.js	iniciarCarrinho()	3560-3670	Estado do carrinho
js/ia.js	InteligenciaFutebol	8-13129	Sistema completo de IA
js/ia.js	_atualizarPlanoTatico()	290-763	Plano coletivo
js/ia.js	atualizarTime()	1325-1895	Entrada por quadro da IA
js/ia.js	_atacarComBola()	4886-7160	Decisão ofensiva do portador
js/ia.js	_melhorPasse()	7608-8650	Escolha de passe
js/ia.js	_defender()	10639-11104	Comportamento defensivo
js/ia.js	_goleiro()	11369-12632	IA do goleiro
js/jogo.js	JogoFutebolWeb	8-36575	Coordenação geral
js/jogo.js	_formacao()	1004-1991	Formações 5 x 5 e 11 x 11
js/jogo.js	_criarTimes()	2076-2309	Instanciação dos jogadores
js/jogo.js	novaPartida()	2539-2601	Início da partida
js/jogo.js	loop()	5696-5760	requestAnimationFrame
js/jogo.js	atualizar()	5761-6521	Processamento geral
js/jogo.js	acaoPasse()	6944-7053	Entrada de passe
js/jogo.js	passar()	8900-9507	Execução do passe
js/jogo.js	cruzar()	9765-10006	Cruzamento
js/jogo.js	chutar()	10007-10427	Finalizações
js/jogo.js	reporGoleiro()	10428-10790	Reposição do goleiro
js/jogo.js	tentarDesarme()	10791-11287	Desarme em pé
js/jogo.js	tentarCarrinho()	11288-11946	Carrinho e contato
js/jogo.js	_tratarInfracao()	11947-12170	Falta e vantagem
js/jogo.js	_aplicarCartao()	12281-12727	Disciplina
js/jogo.js	_verificarNumeroMinimoJogadores()	13552-13740	Limite após expulsões
js/jogo.js	_defesasGoleiro()	15409-15845	Defesas
js/jogo.js	_interceptacoesAtivas()	15846-16311	Interceptações
js/jogo.js	_cabecadas()	16833-17716	Jogo aéreo
js/jogo.js	_estaImpedido()	18479-18638	Impedimento
js/jogo.js	_verificarRegras()	18639-19177	Saídas e regras
js/jogo.js	_gol()	19178-19492	Gol e transição
js/jogo.js	_organizarReinicio()	20257-22263	Bola parada

js/jogo.js	_montarBarreira()	22463-22830	Barreira
js/jogo.js	_cobrarEscanteio()	24167-24298	Escanteio
js/jogo.js	_cobrarPenaltiIA()	24438-24693	Pênalti da IA
js/jogo.js	_atualizarCamera()	26465-26833	Câmera
js/jogo.js	teclaPressionada()	26834-27309	Ações por teclado
js/jogo.js	_desenharPartida()	29889-30392	Composição do quadro
js/jogo.js	_projetarCampo()	30393-30511	Perspectiva 2.5D
js/jogo.js	_desenharEstadio()	30666-31425	Estádio
js/jogo.js	_desenharJogador()	32749-34745	Jogadores
js/jogo.js	_desenharHud()	34992-35728	HUD
js/jogo.js	_desenharMiniMapa()	35729-36001	Minimapa

46. Como estudar e ampliar o código

Uma forma eficiente de compreender o projeto é começar por `configuracoes.js`, depois estudar Bola e Jogador, seguir para a entrada principal `atualizarTime()` da IA e, por fim, acompanhar o loop de `JogoFutebolWeb`. A partir daí, escolha um fluxo específico - por exemplo passe, falta ou gol - e siga as funções chamadas até a renderização.

Ao adicionar um novo recurso, mantenha a separação existente. Valores globais devem ir para `CFG`; propriedades físicas pertencem às entidades; decisões coletivas ou individuais da CPU pertencem à `InteligenciaFutebol`; regras, placar, reinícios e desenho ficam coordenados por `JogoFutebolWeb`; eventos do navegador permanecem em `app.js`.

Essa organização reduz efeitos colaterais. Um novo tipo de finalização, por exemplo, pode ser composto por: uma nova ação em `JogoFutebolWeb`, parâmetros físicos enviados à Bola, critérios de escolha na IA, animação em Jogador e representação visual no desenho. Cada camada recebe apenas a responsabilidade que lhe corresponde.